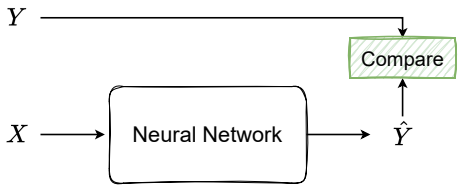
DEEP LEARNING

Unsupervised Learning Part I

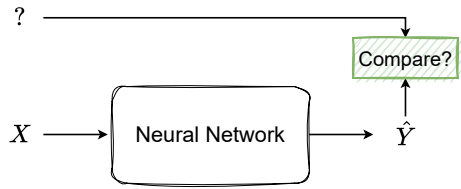
Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)
University of Bergamo

Learning Paradigms



Supervised Learning



Unsupervised Learning

Unsupervised Learning

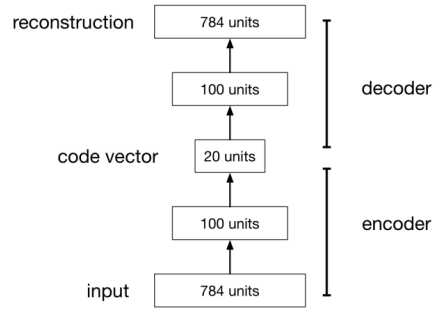
- Unsupervised learning is a type of machine learning that learns from data **without** human supervision, i.e., without target labels.
- Some goals and applications:
  - Finding hidden structures in data
  - Learning representations
  - Dimensionality reduction (e.g., PCA)
  - Data Compression
  - Density estimation
  - Generating new examples
  - Many more...

Table of Contents

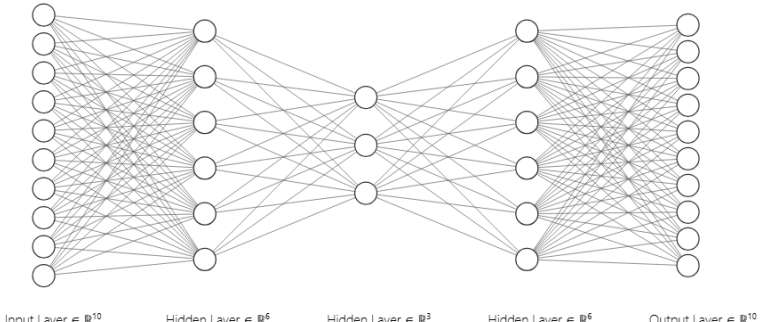
Autoencoder

Autoencoder

- An autoencoder (AE) is a (neural) network that is trained to predict the input, i.e., input $\approx$ output.
- An AE consists of two parts:
  - **Encoder** that maps the input to a hidden representation (also called code vector or latent representation).
  - **Decoder** that maps the hidden representation back to the input space.
- **Key idea**: The AE must learn to encode important attributes in the code vector to faithfully reconstruct the input.



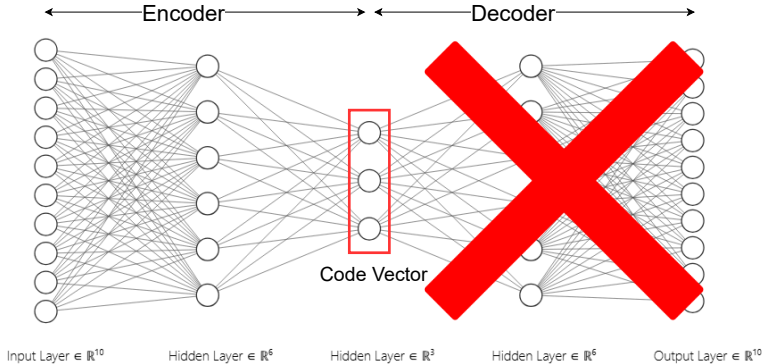
Fully-Connected Autoencoder



- Let us denote the input as $\mathbf{x}$ and the reconstruction (or output) as $\hat{\mathbf{x}}$.
- We use Mean Squared Error Loss to train the AE, like any regular neural network:

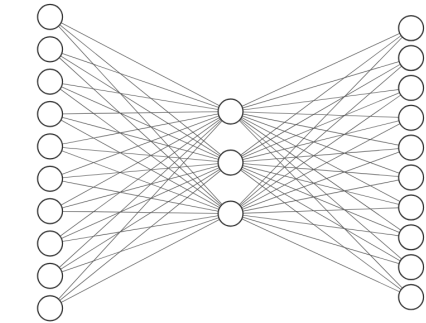
$$\mathcal{L}_{\text{MSE}}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 = \sum_i (x_i - \hat{x}_i)^2$$

Fully-Connected Autoencoder: Dimensionality Reduction



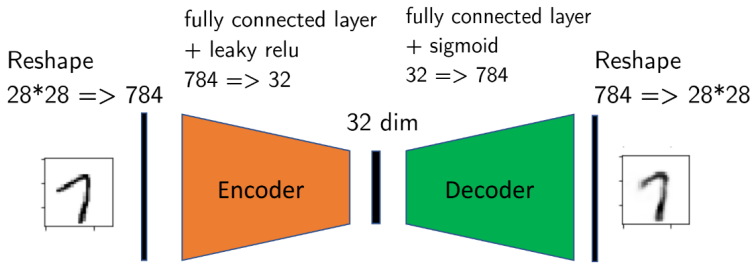
- Once the AE is trained, we discard the decoder, and use the encoder to obtain the lower-dimensional representation of the input $\rightarrow$ dimensionality reduction.
- As the network tapers in the middle, we also call the middle layer bottleneck.

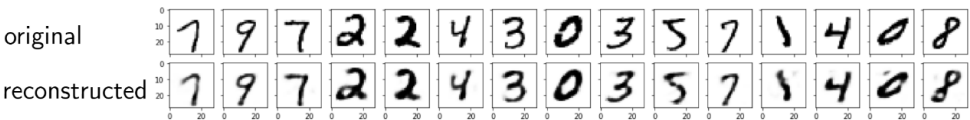
Connection to PCA



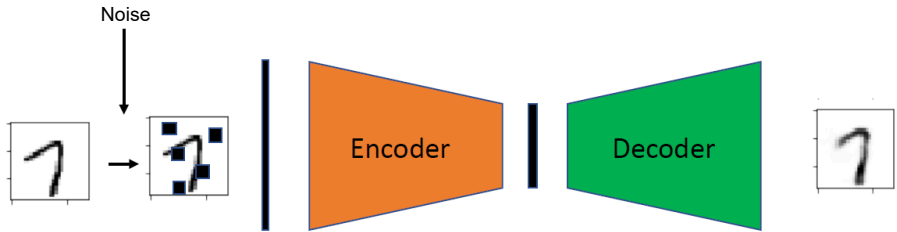
- If we do not use any non-linear activation function in the hidden layers and minimize the MSE loss, then it is similar to PCA.
- The difference being that the latent dimensions will not necessarily be orthogonal to each other.

A Simple Fully-Connected Autoencoder





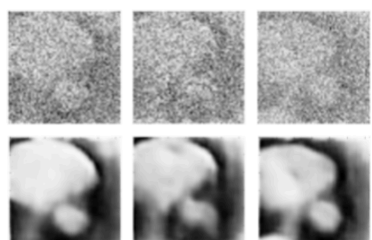
Denoising Autoencoder



- We can train an AE to learn to denoise input, by giving an input $\tilde{\mathbf{x}}$ *corrupted* by noise and aiming to reconstruct the *uncorrupted* (or clean) input $\mathbf{x}$.
- Examples of noise include:
  - Gaussian noise: $\tilde{\mathbf{x}} = \mathbf{x} + \mathcal{N}(0, \sigma^2\mathbf{I})$.
  - Randomly masking out some features, i.e., setting some input features to zero.
  - Adding salt-and-pepper noise, which is randomly setting some input features to either min or max value.

Denoising Autoencoder

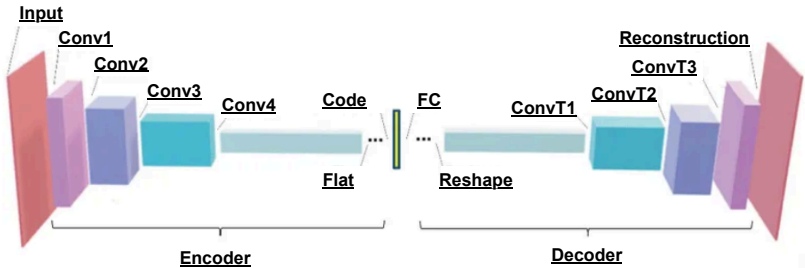
- **Key idea**: The corruption process forces the network to learn meaningful features that capture the essence of the input, and not just memorize the input.
- Has some key advantages over a regular AE:
  - Improves robustness to input perturbations.
  - Helps prevent overfitting.



Convolutional Autoencoder

- When it comes to working with image data, a fully-connected AE shares the same limitations as a fully-connected neural network:
  - Ignores spatial structure.
  - Don't scale well to large images (too many parameters).
  - Needs to learn local patterns like edges and textures from scratch (wasteful computing).
- The solution is to replace the fully-connected layers with convolutional layers. Such an autoencoder is called **Convolutional Autoencoder**.
- Conv AE are better suited for image data.

Convolutional Autoencoder

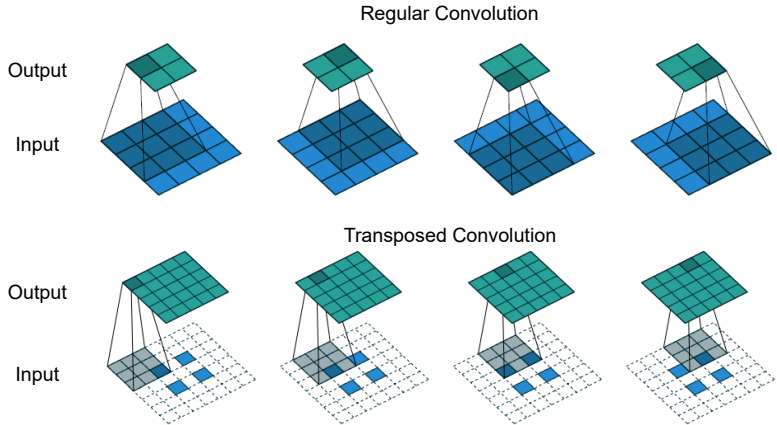


Overview of a Conv AE architecture:

- **Encoder**: Input $\rightarrow$ Conv $\rightarrow$ ReLU $\rightarrow$ Pooling (repeat).
- **Bottleneck** (Latent Code): can be spatial feature maps or flatenned features.
- **Decoder**: Transposed Conv $\rightarrow$ ReLU (repeat).

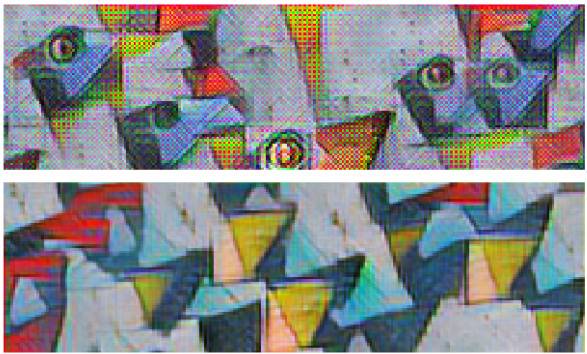
Transposed Convolution

- **Transposed Convolution** is used in the decoder to upsample feature maps – the opposite of standard convolution. It pads zeros and uses the Conv operation.
- Increases spatial resolution (e.g., from $7 \times 7 \rightarrow 14 \times 14$)



Transposed Convolution and Checkerboard Artifacts

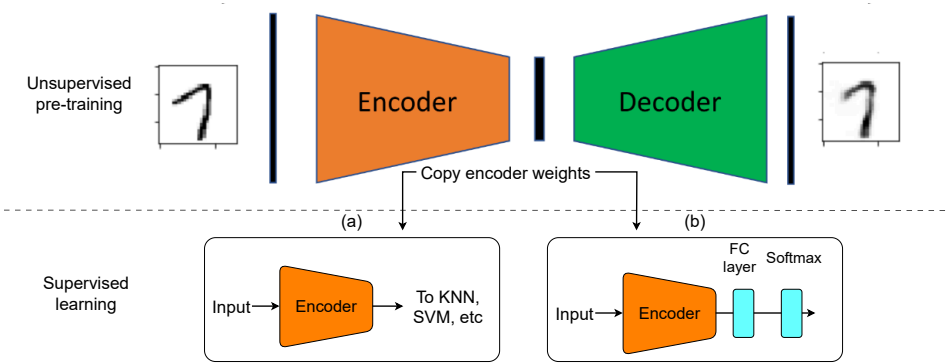
- Transposed convolution (or "deconvolution") can produce checkerboard artifacts.
- It is recommended to replace the transposed convolution by upsampling (interpolation) followed by regular convolution, denoted as resize-convolution.



Using deconvolution.
Heavy checkerboard artifacts.

Using resize-convolution.
No checkerboard artifacts.

Autoencoders: Learning Representations

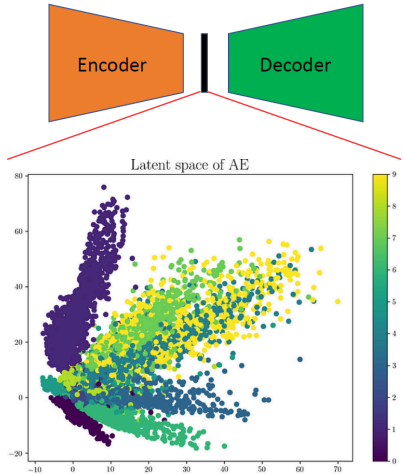


- After unsupervised pre-training, an AE can be used as following:
  - Use the latent features as input to ML algorithms such as KNN and SVM.
  - Transfer learning, where we add a classifier (FC layers) on top of the encoder and fine-tune using a small labelled dataset.

Limitations of Autoencoders

Using MNIST digits dataset as an example:

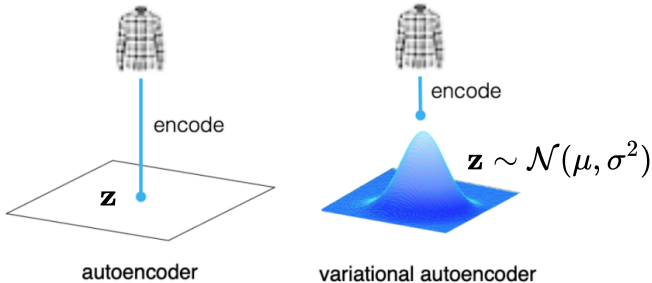
- Digits in the latent space tend to form distinct clusters.
- Large parts of the latent space don't correspond to real data and can generate meaningless outputs when passed through the decoder.
- The latent space is not regularized, so only specific regions are useful for generation.
- AEs are best suited for compression, not for generating new, valid samples.



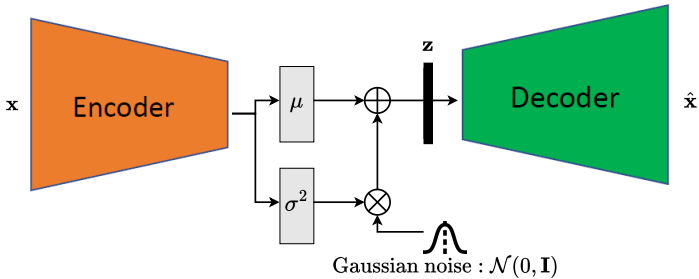
Variational Autoencoder

Variational Autoencoder (VAE)

- **Variational Autoencoders** (VAEs) regularize the latent space, unlike standard AEs, enabling generative capability across the entire space.
- Instead of direct latent vectors, the encoder outputs **parameters of a probability distribution** (e.g., mean and variance).
- A **constraint** (KL divergence) forces these distributions to resemble a Gaussian distribution, ensuring a smooth, continuous latent space.



Variational Autoencoder: Architecture



- The architecture of VAE is similar to an AE, with a key difference in the bottleneck:
  - Encoder outputs two vectors: mean μ, and variance σ^2.
  - Draw the latent vector from the Gaussian distribution: $\mathbf{z} \sim \mathcal{N}(\mu, \sigma)$. In practice we use a reparameterization trick: $\mathbf{z} = \mu + \epsilon\sigma, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
  - Decode $\mathbf{z}$ to get the reconstructed input.

Variational Autoencoder: Training

- The goal of VAE is not only to reconstruct the input, but also to regularize the latent space, such that it follows two essential properties:
  - **Continuity**: Nearby points in latent space should yield similar content when decoded.
  - **Completeness**: Any point sampled from the latent space should yield meaningful content when decoded.
- We train a VAE like a regular AE, but using the sum of two losses:
  - Reconstruction loss (e.g., MSE Loss), as used in AE:

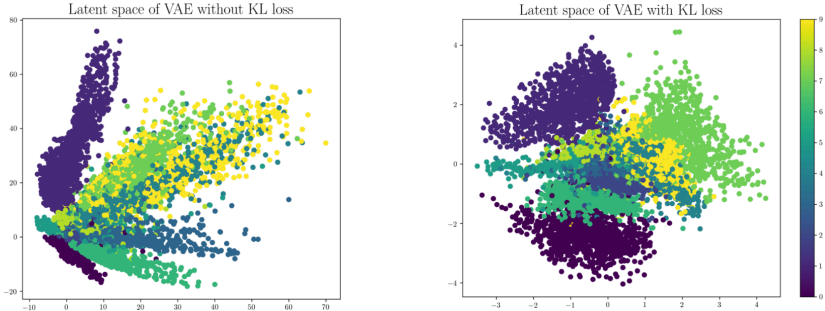
$$\mathcal{L}_{\text{MSE}}(\mathbf{x}, \hat{\mathbf{x}}) = \|\mathbf{x} - \hat{\mathbf{x}}\|^2$$

  - Regularization loss measuring distance (KL divergence[1]) from the latent distribution predicted by the encoder and standard Gaussian distribution:

$$\mathcal{L}_{\text{reg}} = D_{\text{KL}}\big(\mathcal{N}(\mu, \sigma) \| \underbrace{\mathcal{N}(\mathbf{0}, \mathbf{I})}_{\text{prior}}\big)$$

[1] KL divergence measures how much one probability distribution is different from another

Variational Autoencoder: Latent Space



- VAE training balances reconstruction and KL divergence losses to shape a smooth, continuous latent space.
- The latent space approximates a standard Gaussian distribution, ensuring even spread and no large gaps between clusters.
- Clusters of similar data (e.g., digits) may overlap, allowing for smooth transitions.

Variational Autoencoder: Inference

- Once trained, the decoder of the VAE can be used to generate new samples by sampling from the latent space. Works as follows:
  - Sample a latent vector: $\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$.
  - Get a brand new sample by decode the latent vector with the decoder: $\hat{\mathbf{x}} = \text{Dec}(\mathbf{z})$.
- The VAE's latent space is continuous and regularized, so every point sampled from it should decode into something realistic. Randomly generated examples below.

Variational Autoencoder: Traversing the Latent Space

- **Goal**: Generate smooth transitions between two data points (e.g., images) by interpolating their latent representations. Works as follows:
  - Encode two inputs:
    $$\mathbf{z}_1 = \text{Enc}(\mathbf{x}_1), \mathbf{z}_2 = \text{Enc}(\mathbf{x}_2)$$
  - Linearly interpolate in latent space:
    $$\mathbf{z} = \alpha\mathbf{z}_1 + (1-\alpha)\mathbf{z}_2, \alpha \in [0,1]$$
  - Deocde interpolated latent code to get new data: $\hat{\mathbf{x}} = \text{Dec}(\mathbf{z})$

Interpolating between black hair and blonde hair

Interpolating between faces with and without sunglasses

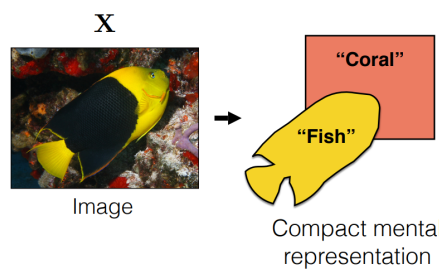
Representation Learning

Representation Learning

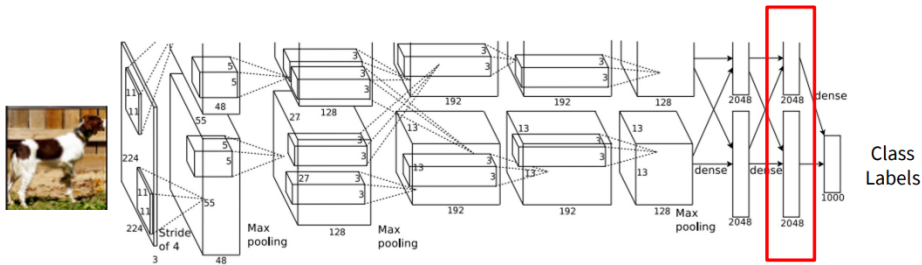
Representation learning involves automatically discovering useful features or representations from raw data to improve performance on tasks like classification.

Good representations are:

1. Compact (*minimal*)
2. Explanatory (*sufficient*)
3. Disentangled (*independent factors*)
4. Interpretable
5. Make subsequent problem (e.g., classification) solving easy

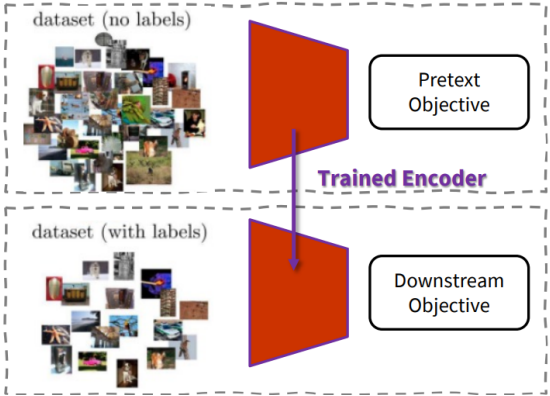


Learning Good Representations



- Good representations (indicated by the red box) can be learned using lots of labelled data (in the order of millions).
  - But this is expensive since it requires manual annotations (or labels).
- Q: What is an alternative to this approach? Meaning can we learn good representations *without* using lots of manually labelled images?

Self-Supervised Learning



- A **pretext task** is a self-supervised objective designed to learn useful features from unlabelled data by solving a simpler, auxiliary problem (requires no manual labels).
- A **downstream task** is an application that you care about (e.g., classifying pipes) and usually you will have a small labelled dataset.

Self-Supervised Pretext Task

Example: learn to predict image transformations / complete corrupted images

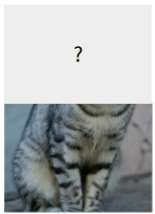
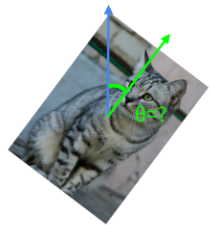


image completion



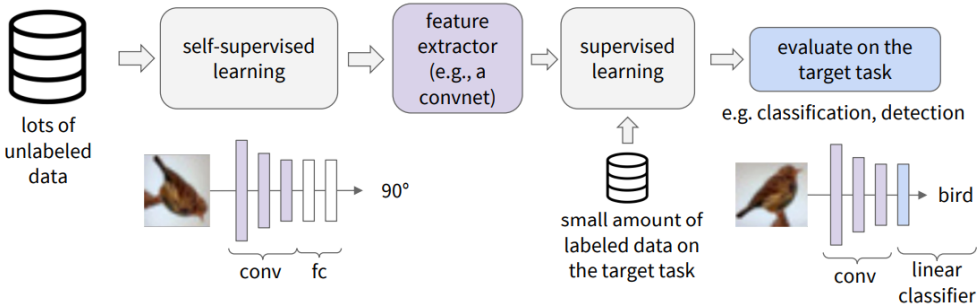
rotation prediction

“jigsaw puzzle”

colorization

1. Solving the pretext tasks allow the model to learn good features.
2. We can automatically generate labels for the pretext tasks.

Evaluating Self-Supervised Learning Methods

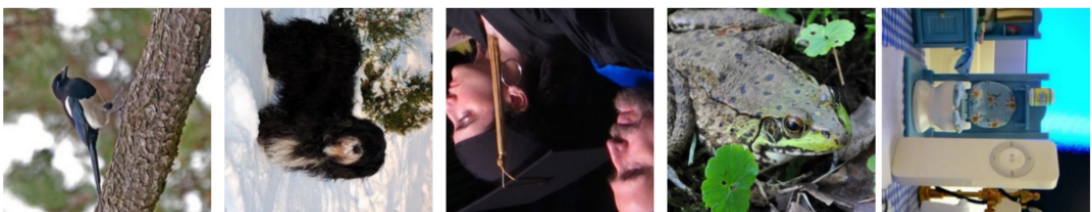


1. Learn good feature representation using a self-supervised pretext task, e.g., predicting image rotations.

2. Attach a shallow network (typically a MLP or linear layer) on top of the feature extractor; and train the shallow network using a small labelled dataset.

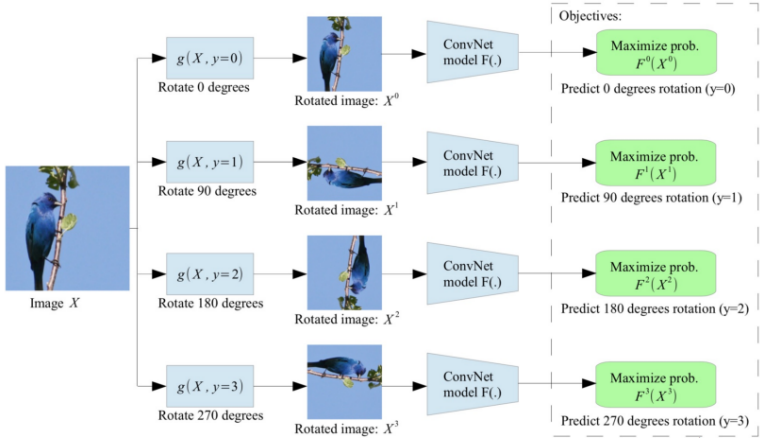
Pretext Task: Predict Rotations

Pretext task: Rotate the image and train the network to predict the rotation applied to the original image.



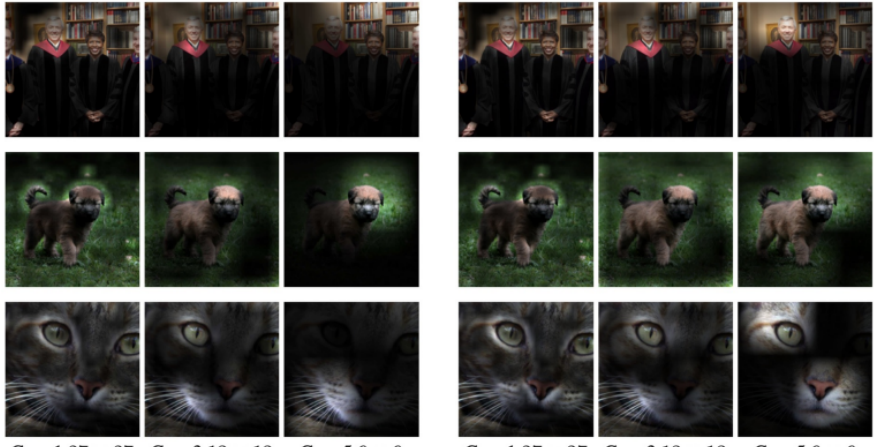
Hypothesis: A model could recognize the correct rotation of an object only if it has the "visual commonsense" of what the object should look like unperturbed (or in upright position).

RotNet: Predict Rotations



- Each rotation angle is assigned a class label (e.g., 90° rotation means Class 1, so on).
- The model learns to predict which rotation is applied (4-way classification problem)

RotNet: Visualizing Learned Representations

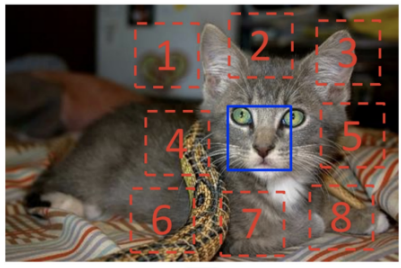


(a) **Attention maps of supervised model**

(b) **Attention maps of our self-supervised model**

Pretext Task: Predict Relative Patch Locations

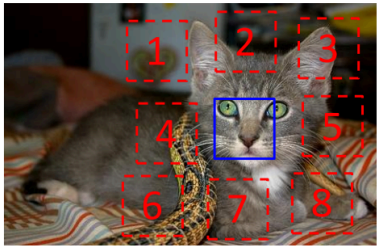
Pretext task: Randomly sampling a patch (blue) and then one of eight possible neighbours (red), guess the spatial configuration (or relative location) of the pair of patches.

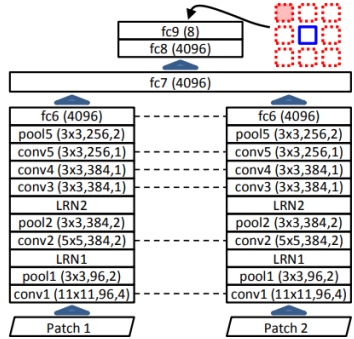


Question: What is the relative location?
Answer: Top left

Hypothesis: A model could successfully guess the relative patch location only if it has learned to recognize the object in the image.

Pretext Task: Predict Relative Patch Locations

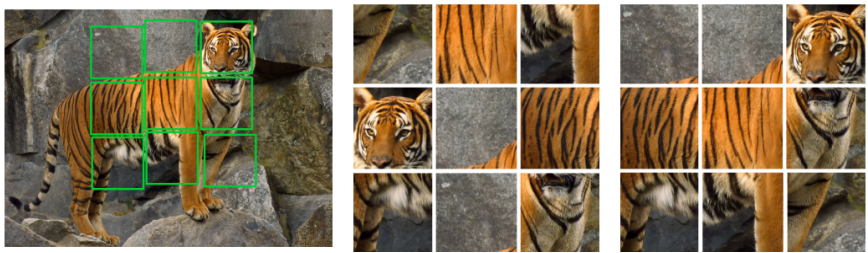




- Each neighbour is assigned a class label (e.g., the top-right location means Class 3).
- The model learns to predict what is the relative location (an 8-way classification problem).
- Note that we have two networks, where the dotted lines denote shared parameters.

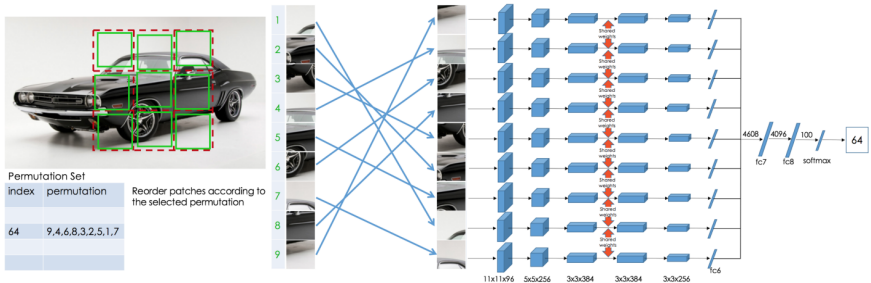
Pretext Task: Solving "Jigsaw Puzzles"

Pretext task: Given a set of patches (shown in green), randomly shuffle them, and ask the model to guess the order in which the patches were shuffled.



Hypothesis: A model can only successfully guess the correct order of shuffling if it understands the object in the image well.

Pretext Task: Solving "Jigsaw Puzzles"



- These 9 tiles are reordered via a randomly chosen permutation from a predefined permutation set and are then fed to the network.
- Each reordering (or shuffling) is assigned a class label (e.g., {9,4,6,8,3,2,5,1,7} is assigned the class label 64).
- The task is to predict the label corresponding to the chosen permutation (a 64-way classification problem).

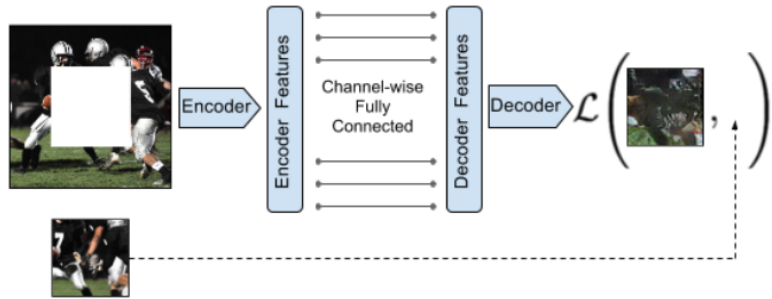
Pretext Task: Predict Missing Pixels (Inpainting)

Pretext task: Natural images, despite their diversity, are highly structured (e.g., the regular pattern of windows on the facade). Given an image with a missing region (the white blank), a model is trained to regress (or inpaint) the missing pixel values.

Input context | Human artist | Inpainting by model

Hypothesis: A model needs a deeper semantic understanding of the scene to inpaint the correct pixel values.

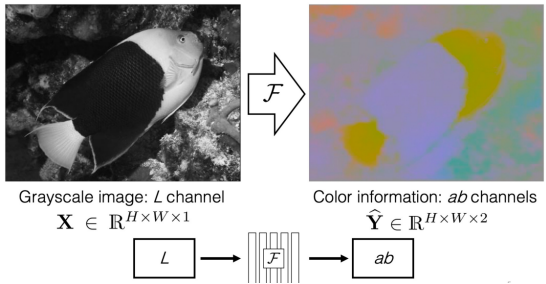
Pretext Task: Predict Missing Pixels (Inpainting)



- The architecture and training technique is similar to autoencoder.
- The target label for the network is the region that is omitted from the input image.
- The image is passed through the encoder to obtain features which are connected to the decoder. The decoder then reconstructs the missing regions in the image.

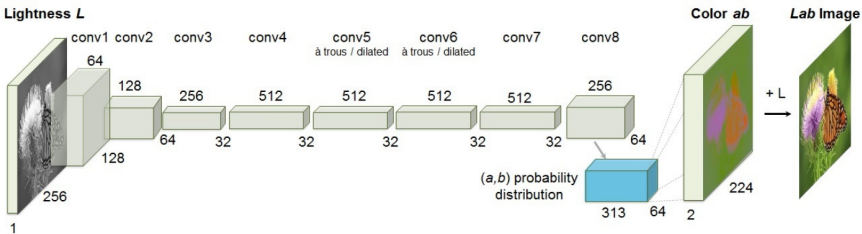
Pretext Task: Image Coloring

Pretext task: Given the grayscale image a model is trained to colorize the grayscale image. More specifically, given the lightness L channel, the model predicts the a and b channels (or the color channels) of the Lab color space.



Hypothesis: To produce a plausible colorization a model must have a good semantic understanding of objects (e.g., sky is usually blue, grass is usually green, and ladybug is almost always red).

Pretext Task: Image Coloring



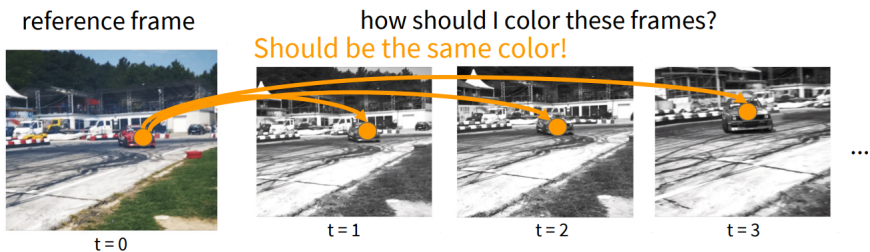
- The network is again similar to an autoencoder.
- The model predicts the ab channels given the L channel as input.
- The L channel is merged with the ab channels to get the colored image as the model prediction. Reconstruction loss is constructed between the predicted color image and the original RGB image.

Colorizing Legacy Black and White Photos

Left to right: photo by David Fleay of a Thylacine, now extinct, 1936; photo by Ansel Adams of Yosemite; amateur family photo from 1956; Migrant Mother by Dorothea Lange, 1936.

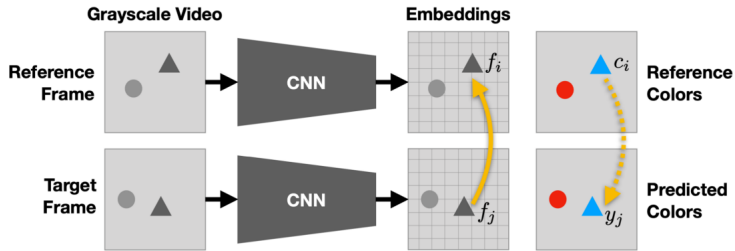
Pretext Task: Video Coloring

Pretext task: Find corresponding pixels (or regions) among nearby frames in a video that belong to the same object and hence must be of the same color value. This time the pretext task is designed for tracking objects across frames (an important task in computer vision).



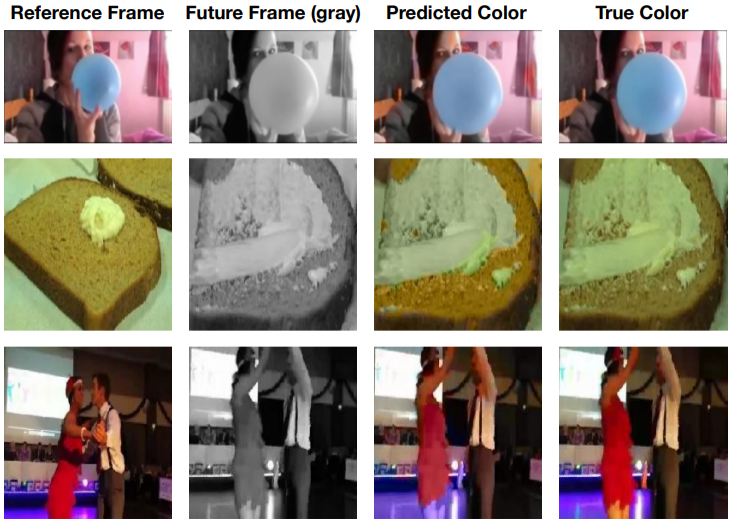
Hypothesis: Learning to color video frames should allow model to learn to track regions or objects without labels.

Pretext Task: Video Coloring

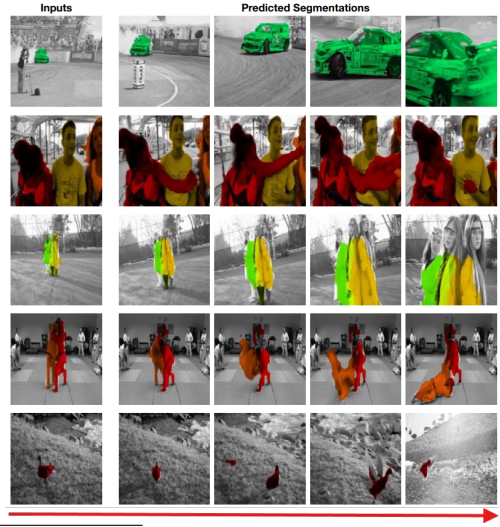


- Attention map on the reference frame: $A_{ij} = \frac{\exp(f_i^\intercal f_j)}{\sum_k \exp(f_k^\intercal f_j)}$. Intuitively, it means how similar a pixel j (in the target frame) is to all the pixels in the reference frame.
- Predicted color = weighted sum of the reference frame colors: $\hat{y}_j = \sum_i A_{ij} c_i$.
- Training loss at the j^{th} pixel is computed between the predicted color and the ground truth color of the target frame: $\mathcal{L}_j(\hat{y}_j, y_j)$.

Pretext Task: Video Coloring



Video Coloring: Tracking Results



[0] https://research.google/blog/self-supervised-tracking-via-video-colorization/

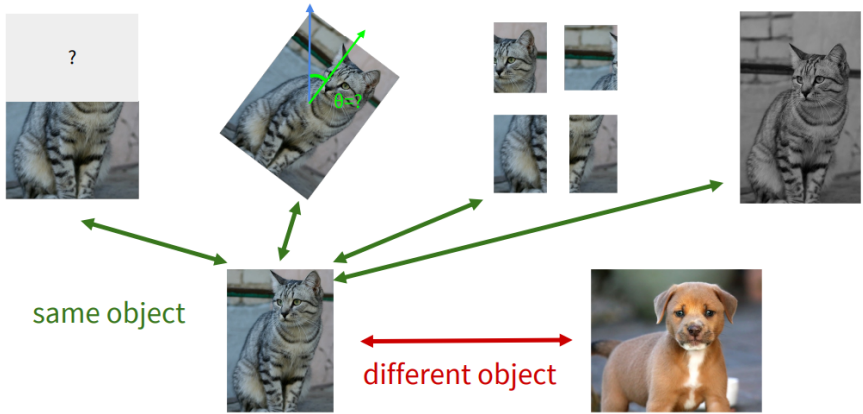
Limitations of Pretext Tasks

While the pretext tasks seems to work well for some tasks, it has some drawbacks:

- Developing individual pretext tasks is tedious.
- The learned representations may not be general.

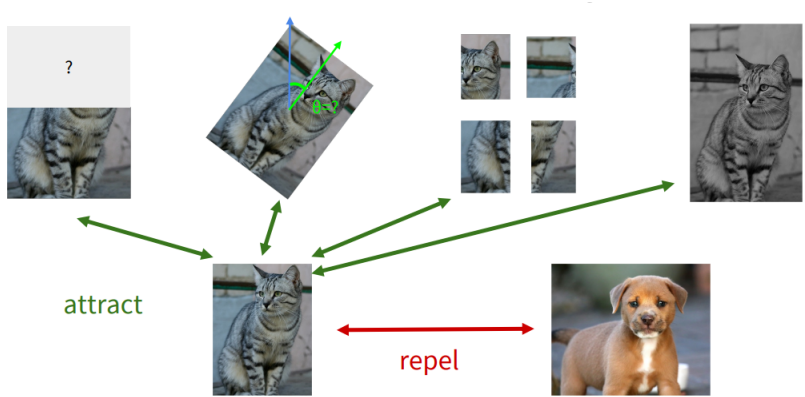
Can we come up with a more general pretext task that can work across both images and videos?

A more General Pretext Task



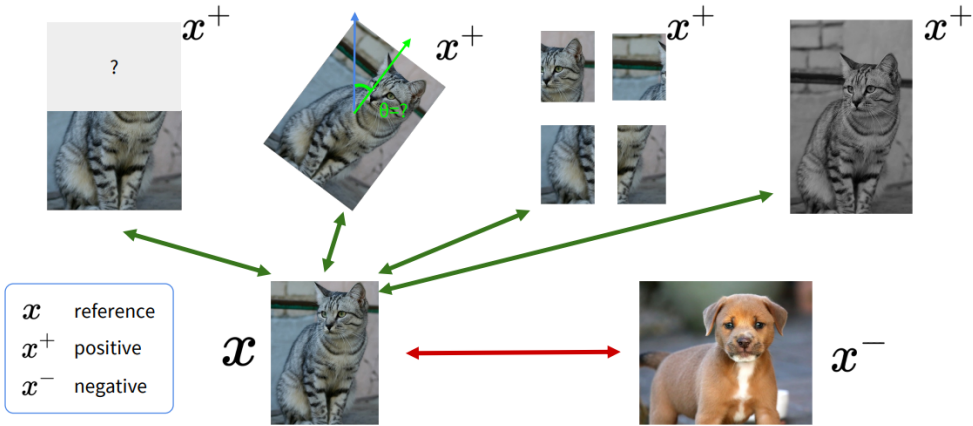
The pretext task is designed using the notion of objects that are the same/similar, and objects that are different/dissimilar.

Contrastive Representation Learning



Key idea: Different crops of an image that belong to the same object should be more similar to each other than the crops coming from a different image in the representation space.

Contrastive Representation Learning



Contrastive Representation Learning

- Formally, we want:

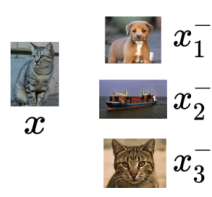
$$\text{score}(f(x), f(x^+)) \gg \text{score}(f(x), f(x^-)),$$

  where x : reference sample, x^+: positive sample, and x^-: negative sample.
- Given a chosen score function, we aim to learn an encoder function f that yields high score for positive pairs (x, x^+) and low scores for negative pairs (x, x^-).

Contrastive Representation Learning

Loss function given 1 positive sample and $N-1$ negative samples:

$$\mathcal{L} = -\log \frac{\overbrace{\exp(s(f(x), f(x^+)))}^{\text{score for the positive pair}}}{\underbrace{\exp(s(f(x), f(x^+)))}_{\text{score for the positive pair}} + \underbrace{\sum_{j=1}^{N-1} \exp(s(f(x), f(x_j^-)))}_{\text{score for negative pairs}}}$$

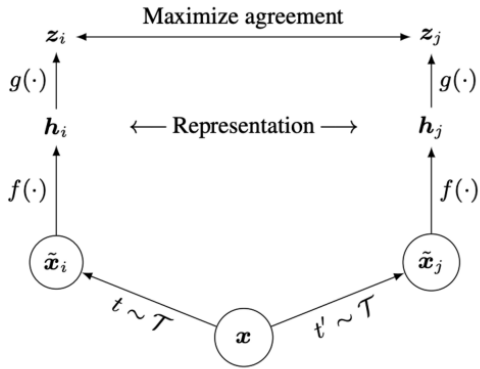


SimCLR: A Simple Framework for Contrastive Learning

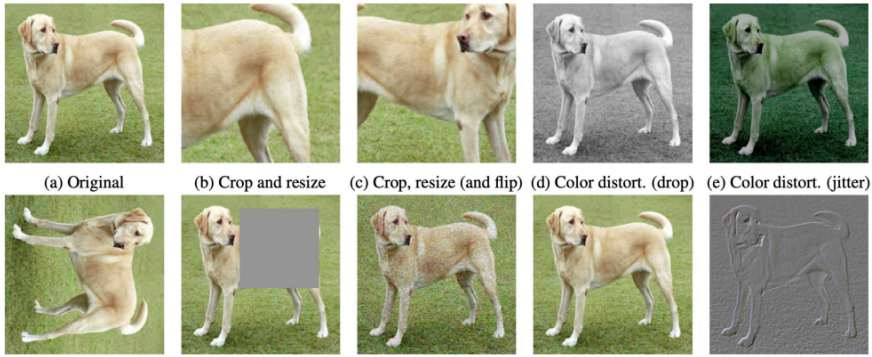
- Cosine similarity as the scoring function:

$$S(u, v) = \frac{u^\intercal v}{\|u\|\|v\|}.$$

- Use a projection network $g(\cdot)$ to project features to a space where contrastive learning is applied.
- Generate positive samples through data augmentation:
  - random cropping, random color, distortion, and random blur.



SimCLR: Augmentations for Generating Positive and Negative Samples



(a) Original (b) Crop and resize (c) Crop, resize (and flip) (d) Color distort. (drop) (e) Color distort. (jitter)

(f) Rotate $\{90°, 180°, 270°\}$ (g) Cutout (h) Gaussian noise (i) Gaussian blur (j) Sobel filtering

Contrastive Learning Frameworks

- The general idea of contrasting between positive and negative samples is a widely used technique for learning feature representations in a self-supervised manner.
- It is used in frameworks such as:
  - MoCo (`https://arxiv.org/abs/1911.05722`)
  - MoCo V2 (`https://arxiv.org/pdf/2002.05709.pdf`)
  - Contrastive Predictive Coding (`https://arxiv.org/abs/1807.03748`)
  - DINO (`https://arxiv.org/abs/2104.14294`)
  - And hundreds...

References and Links

- For additional reading materials refer to the following links
  - "Deep Learning Book for Autoencoders"[2].
  - "Lilian Weng Blog Post for Self-supervised Learning"[3].
- Slide and image courtesy: Fei Fei Li, E. Adeli, S. Scott, and S. Raschka.

[2] https://www.deeplearningbook.org/contents/autoencoders.html
[3] https://lilianweng.github.io/posts/2019-11-10-self-supervised/